

IBM Machine Learning Professional Certificate
Unsupervised Machine Learning - Course Project

A Project Report

on

**Comparative Analysis of Unsupervised
Machine Learning Models for Clustering
on Iris Dataset**

by

Aryan Agarwal

August, 2025

Table of Contents

1. Introduction and Summary of the Dataset	1
2. Data Cleaning and Preprocessing	3
3. K-Means Clustering	5
4. Gaussian Mixture Models	7
5. Agglomerative Clustering	8
6. DBSCAN	9
7. Key Findings and Insights	11
8. Conclusion	12

1. Introduction and Summary of the Dataset

1.1 About the Project

In this project, the aim was to build the best possible unsupervised machine learning model on a given dataset. Specifically, the objective was to cluster the dataset into multiple clusters using the features of the dataset. We followed a sequential process to build the model, and the steps included preprocessing the data, including cleaning and visualizations to understand the dataset, after which we tested various models such as K-means clustering, gaussian mixture models, agglomerative clustering and DBSCAN; in order to find the best possible model.

Each model that was built was scored using the silhouette score metric from the `sklearn.metrics()` library. The scores were stored in a separate dataframe, to be analyzed at the end of the project to determine and compare the performances of different models.

1.2 Information about the Dataset

For this project, the dataset chosen was the Iris Dataset, available on Kaggle. This dataset lists basic information about flowers of the iris flower family, namely the petal and sepal lengths and widths.

The dataset also includes a column specifying the species of the flower. We shall not be using this column while building the model. However, the aim shall ideally be to cluster the data points according to their species.

The dataset was imported from the following URL:

<https://www.kaggle.com/datasets/himanshunakrani/iris-dataset>

1.3 Additional Information

This analysis was performed on the web-based Jupyter Notebook IDE, using the Python (Pyodide) kernel.

The coding was done in Python 3.13.3, employing the following libraries for various steps and processes:

- Pandas

- NumPy
- Matplotlib
- Seaborn
- SciPy
- Scikit-learn

1.4 Dataset Summary

The dataset spans 150 rows, each representing a single flower, and 5 columns, each representing a certain detail about the flower. It consists of 1 categorical (object) and 4 numerical (float) columns, as follows:

```
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null    float64
1   sepal_width     150 non-null    float64
2   petal_length    150 non-null    float64
3   petal_width     150 non-null    float64
4   species         150 non-null    object
dtypes: float64(4), object(1)
```

The numeric columns have the following data distribution:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

2. Data Cleaning and Preprocessing

2.1 Cleaning and Visualizations

Before training any models on our dataset, the raw data was cleaned and processed to ensure that it is fit to be passed into the various different models to be trained on it ahead.

As can be seen above, there are no null values in any of the columns, so there was no need to perform any cleaning to fill or remove any data points.

The next step was to visualize the correlations between different features, using the `.heatmap()` function of Seaborn:



2.2 Feature Scaling

Next, we checked if any of the numerical values were skewed and whether they required log transformation to be performed on them. However, the distributions were sufficiently normal.

```
sepal_width      0.334053
sepal_length     0.314911
petal_length     0.274464
petal_width      0.104997
```

Now that the data was cleaned, the next step was to scale the data to ensure that models such as K-means and DBSCAN would not weigh feature importances differently due to scale. This was achieved using the `StandardScaler()` function of scikit-learn.

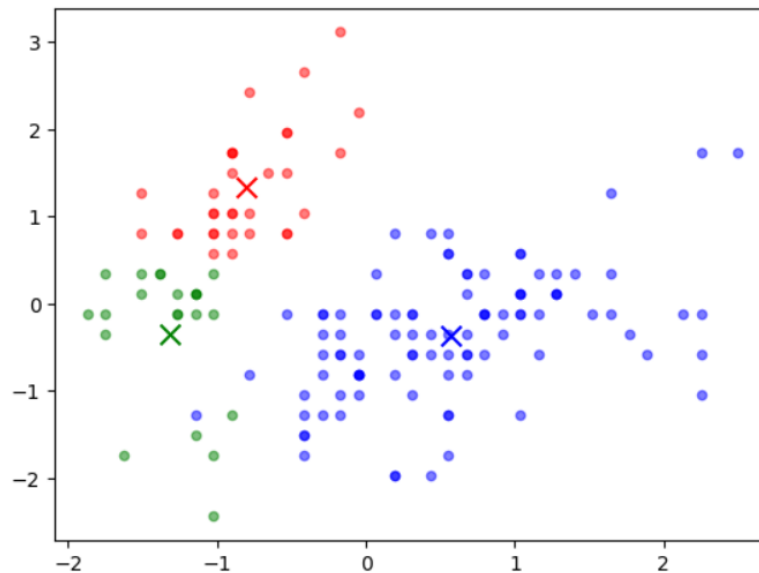
After that, an empty dataframe was initialized to store the scores of the different models we trained, containing columns for the model name, number of clusters, and the silhouette score.

Lastly, we defined multiple functions to visualize the output of every clustering model as 2D scatterplots.

With this, the preprocessing steps were complete, and the dataset was ready to train the first model.

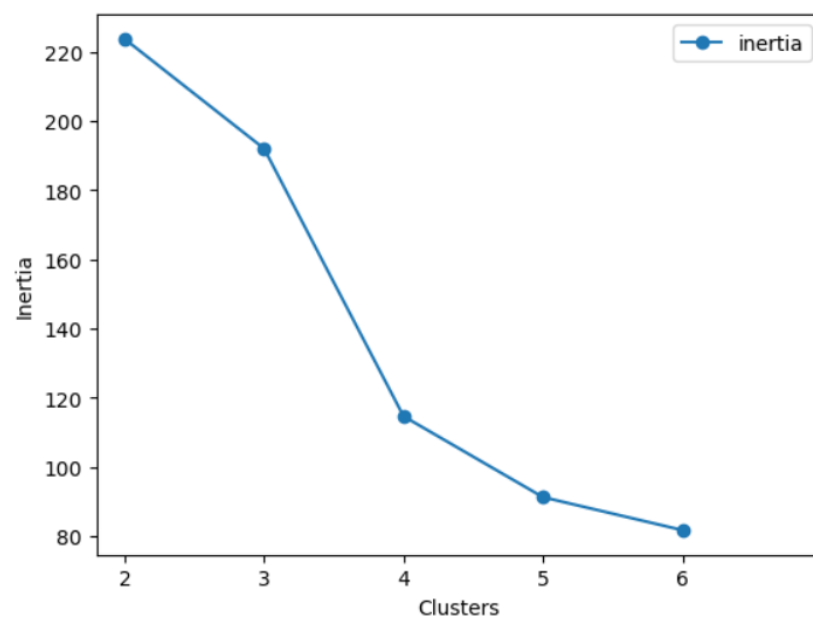
3. K-Means Clustering

The first model that we built was a K-means clustering model, using the `KMeans()` function of `scikit-learn`. The number of clusters was set to 3. The resulting plot was as follows:

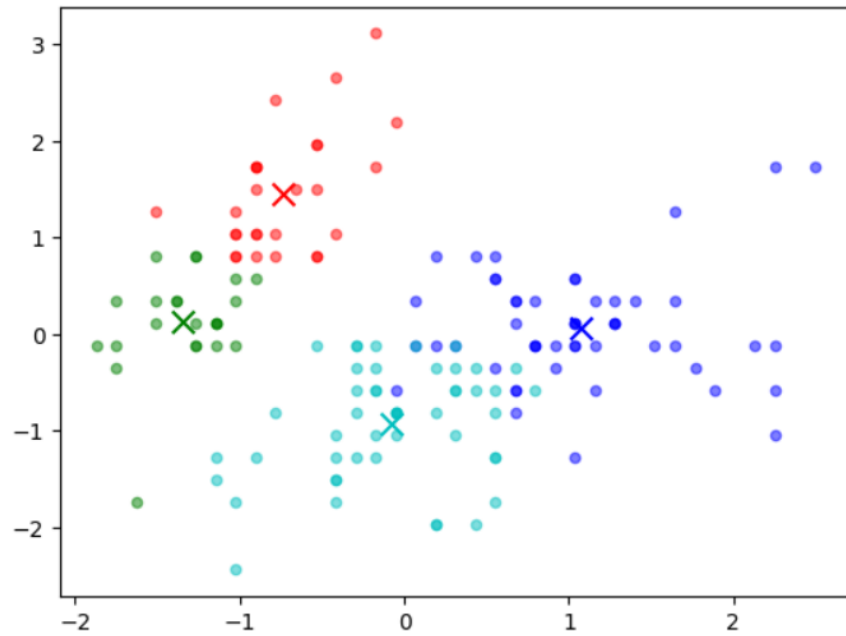


Next, we tuned the model hyperparameters by trying multiple different values for `n_clusters`, to find the best value. The models' silhouette score as well as inertia was stored in a list.

The plot for inertia values was as follows:

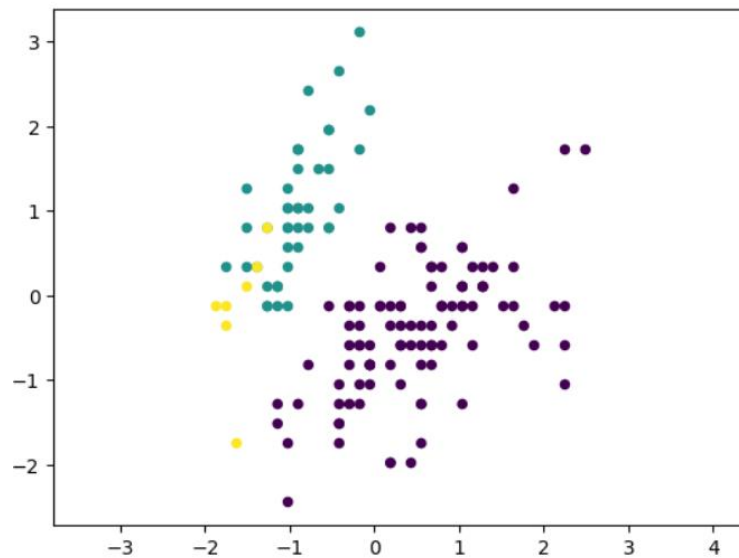


As can be seen, the elbow point was at 4 clusters, so we next visualized the tuned model with 4 clusters. The visualization was as such:



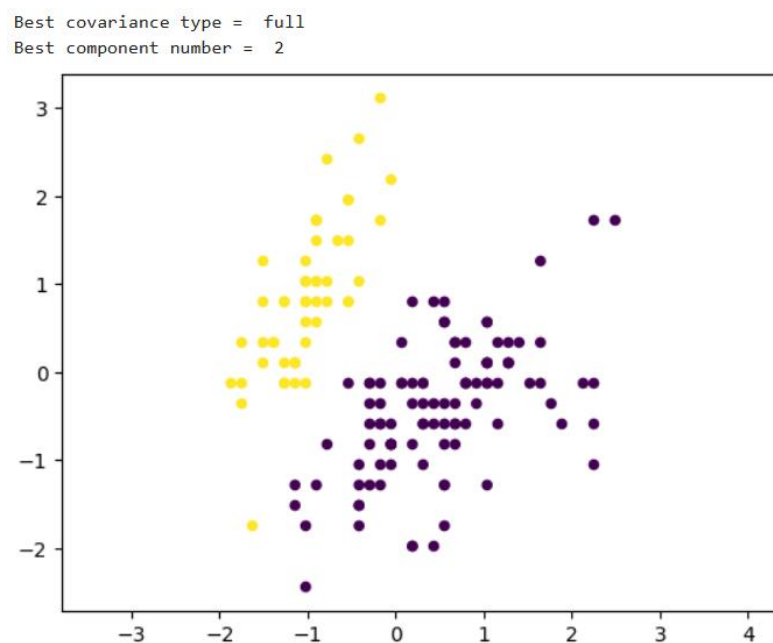
4. Gaussian Mixture Models

The next model that we built was a Gaussian mixture model, using the `GaussianMixture()` function of `scikit-learn`. The 'n_components' was initially set to 3. The resulting plot was as follows:



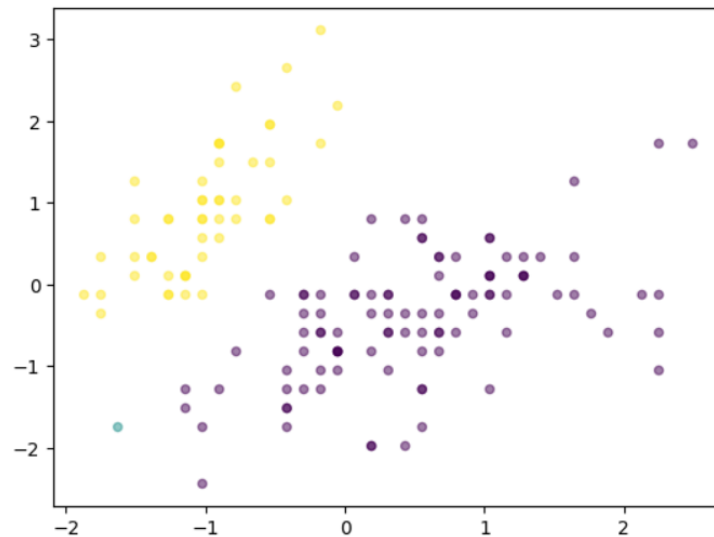
Next, we tuned the model hyperparameters by trying multiple different values for `n_components`, as well as different covariance types, namely 'full' and 'tied'. The best parameters were stored and displayed along with the graph.

The results were as follows:



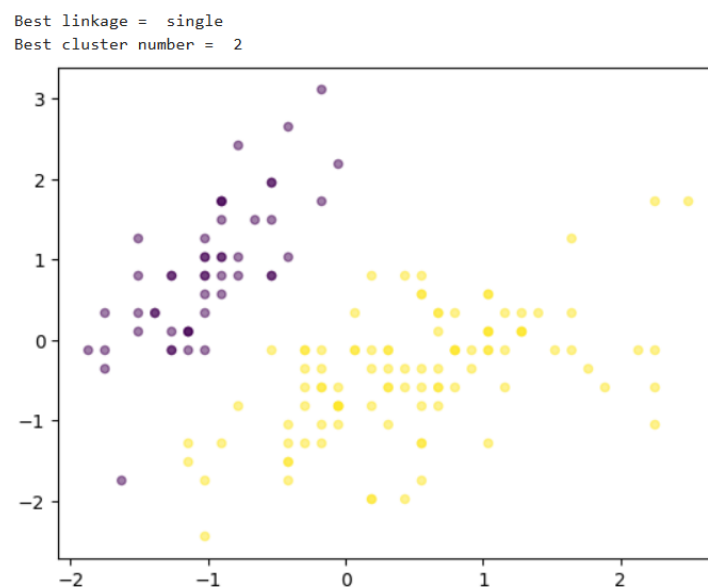
5. Agglomerative Clustering

The next model we built was an agglomerative clustering model, using the `AgglomerativeClustering()` function of `scikit-learn`. The 'n_clusters' was initially set to 3, while 'linkage' was set to 'single'. The resulting plot was as follows:



Next, we tuned the model hyperparameters by trying multiple different values for `n_clusters`, as well as different linkage types, including 'single', 'complete', 'average' and 'ward'. The best parameters were stored and displayed along with the graph.

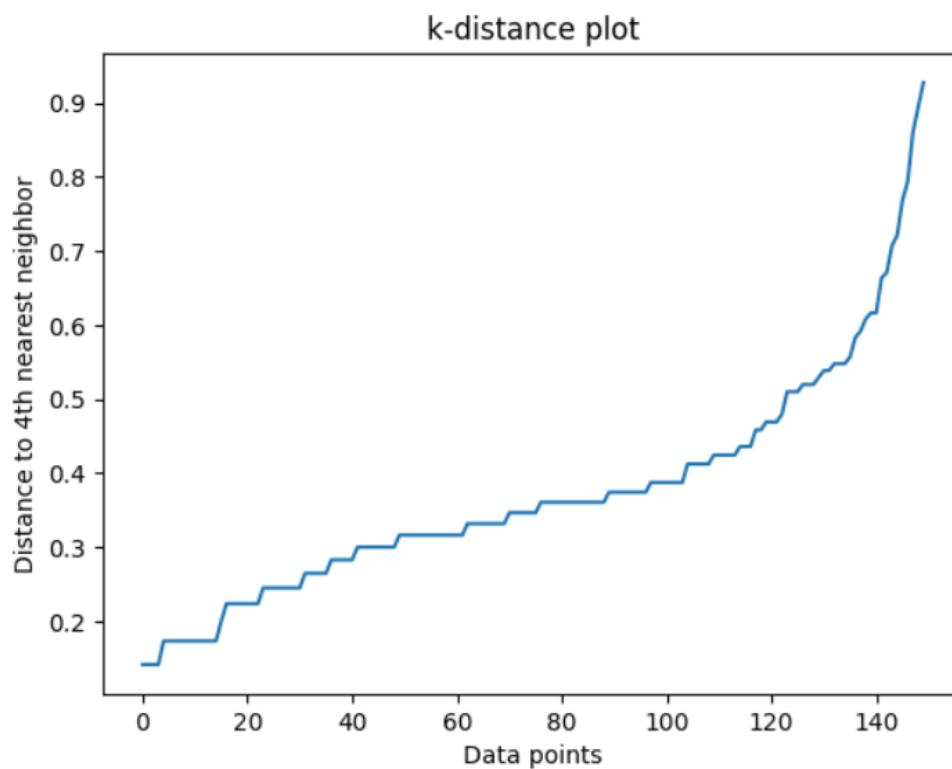
The results were as follows:



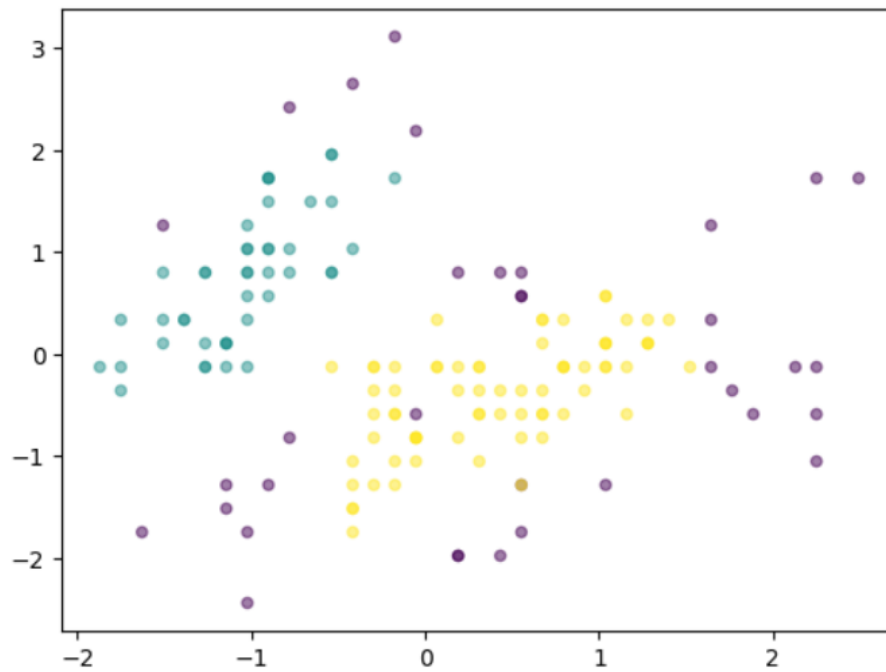
6. DBSCAN

Lastly, we built a DBSCAN (Density-Based Spatial Clustering of Applications with Noise) model, using the `DBSCAN()` function of `scikit-learn`.

To estimate the approximate range of possible values for the 'eps' parameter, we first fit a K-nearest neighbours model on our dataset with 'n_neighbors' set to 4, and graphed the results to find the ideal value of 'eps':

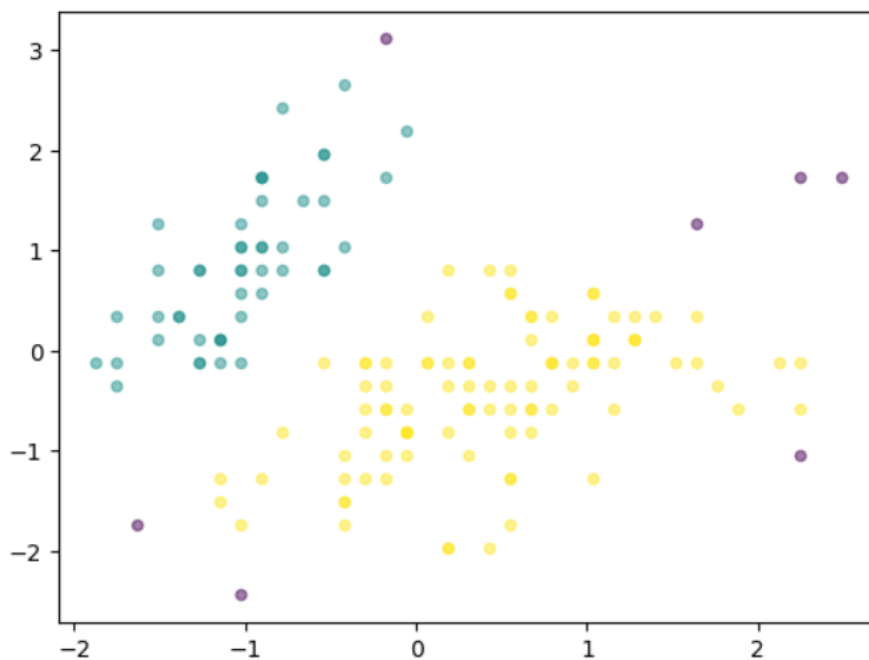


As can be seen, the elbow point lies around 0.5, thus we initially set 'eps' to 0.5 and 'min_samples' to 5 for our first DBSCAN model. The results were as shown:



Next, we iterated through a small range of 'eps' centred around 0.5 as well as multiple 'min_samples' values to tune these parameters, and then displayed the model with the highest silhouette score.

The final graph was as shown, with 'eps' = 0.8 and number of clusters = 3:



7. Key Findings and Insights

7.1 Dataset Takeaways

The progressive models built on the dataset provided some key insights about the data itself as well as the relationships present within the data between different features

While preprocessing, we saw that the dataset was a clean dataset with no missing values and a wide range of features.

A correlation heatmap of the entire dataset showed that the features 'petal_length' and 'petal_width' were highly correlated, and other features such as 'sepal_length' also showed correlation with the aforementioned features.

7.2 Model Performance Summary

Thereafter, a total of 8 different models were trained and tested on this dataset. These models were scored primarily using the silhouette score metric, the results of which are displayed below:

	Model	Clusters	Silhouette Score
0	K-means	3	0.478724
1	K-means	4	0.387247
2	Gaussian Mixture Model	3	0.409184
3	Gaussian Mixture Model	2	0.580184
4	Agglomerative Clustering	3	0.502877
5	Agglomerative Clustering	2	0.580184
6	DBSCAN	3	0.349198
7	DBSCAN	3	0.521564

As can be seen from the above table, the best performing model were the Gaussian Mixture and Agglomerative Clustering models with tuned hyperparameters, with both models achieving a silhouette score of 0.580184.

This finding can be used to infer that the clusters in our dataset may not necessarily spherical and/or dense, but instead could be elliptical and possibly overlapping. Furthermore, correlation between features could be high as K-means performed worse than covariance-aware models like GMM.

8. Conclusion

8.1 Summary

In conclusion, this project proved to be an insightful and comprehensive deep-dive into exploring different clustering models, how they are implemented, and their strong and weak points.

Furthermore, it also revealed key insights about the 'Iris Flowers' dataset, and the relationships and correlations between its features.

In this project, we started off by choosing and importing the 'Iris Flowers' dataset, along with all the libraries that would be used throughout the project. Then the dataset was pre-processed, which involved scaling the dataset, and visualizing the correlations between the features with a heatmap. Then, after defining some helper functions to plot the results of our models, the first model was trained on it. Over the course of the project, 8 different models were trained including the basic and tuned versions of K-means clustering, Gaussian Mixture models, agglomerative clustering, and DBSCAN. Lastly, all models were compared using their silhouette score, and key insights were summarized and presented.

8.2 Next Steps

With the modelling and analysis now complete, the possible next steps that we can take include:

- Exploring the covariance matrix from the GMM model, and the dendrogram from the Agglomerative Clustering model, to see how features interacted within the clusters.
- Compare the clusters to the actual labels from the dropped 'species' column, to see how accurately the clusters predicted the species of flower.
- Try other advanced models such as HDBSCAN, spectral clustering, and self-organizing maps.

These steps will ensure that the takeaways from this modelling project are optimally used, and prove beneficial in practical applications further down the line.